Ministerul Educației și Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică

Raport
la
Lucrarea de laborator Nr. 5

Disciplina: Tehnici de programare

Tema: "funcții, metode și importuri pe web"

A efectuat : st. gr. SI–262
Inculeț Andrian

A verificat : asistent universitar
Dumitrașcu Marius

Chișinău – 2026

SARCINA LUCRĂRII

Partea 1: Importuri (exercițiile 1–8)

Scop: învățați diferitele moduri de a aduce cod în program. Fișier: `part1_imports.py`.

Exercițiul 1. Primul import. Scrieți `import requests` și afișați `requests.__version__`. Apoi scrieți `import urllib.request`. Într-un comentariu, explicați de ce `requests` a avut nevoie de `pip install`, iar `urllib` nu.

Exercițiul 2. Două stiluri de import. Descărcați pagina `BASE_URL` cu `requests.get()` și afișați codul de stare. Apoi rescrieți același script folosind `from requests import get`. Într-un comentariu, scrieți câte un avantaj al fiecărui stil.

Exercițiul 3. Alias-uri. Importați requests cu un alias: `import requests as rq`. Descărcați pagina principală cu `rq.get()`. Într-un comentariu, spuneți când un alias face codul mai ușor de citit și când îl face mai greu de citit.

Exercițiul 4. Doar biblioteca standard. Descărcați pagina principală folosind doar `urllib.request.urlopen()`. Afișați atributul `.status` al răspunsului și primele 200 de caractere din corp.

> **Indiciu:** `.read()` returnează octeți (bytes). Folosiți `.decode("utf-8")` ca să îi transformați într-un șir de caractere.

Exercițiul 5. Priviți în interiorul unui modul. Rulați `print(dir(requests))`. Alegeți trei nume din rezultat și scrieți, într-un comentariu, dacă fiecare este o funcție, o clasă sau un modul.

Exercițiul 6. Citiți documentația. Rulați `help(requests.get)`. Găsiți parametrul care setează timpul maxim de așteptare și folosiți-l într-o cerere.

Exercițiul 7. Cronometrarea unei cereri. Importați `time` și măsurați cât durează o cerere către pagina principală, apelând `time.perf_counter()` înainte și după. Comparați rezultatul cu `response.elapsed`.

Exercițiul 8. Când un import eșuează. Scrieți un script care încearcă `import bs4` într-un bloc `try` / `except ImportError`. Dacă modulul lipsește, afișați: `Instalați modulul cu: pip install beautifulsoup4`.

Nota: Am efectuat copy-paste la conditie

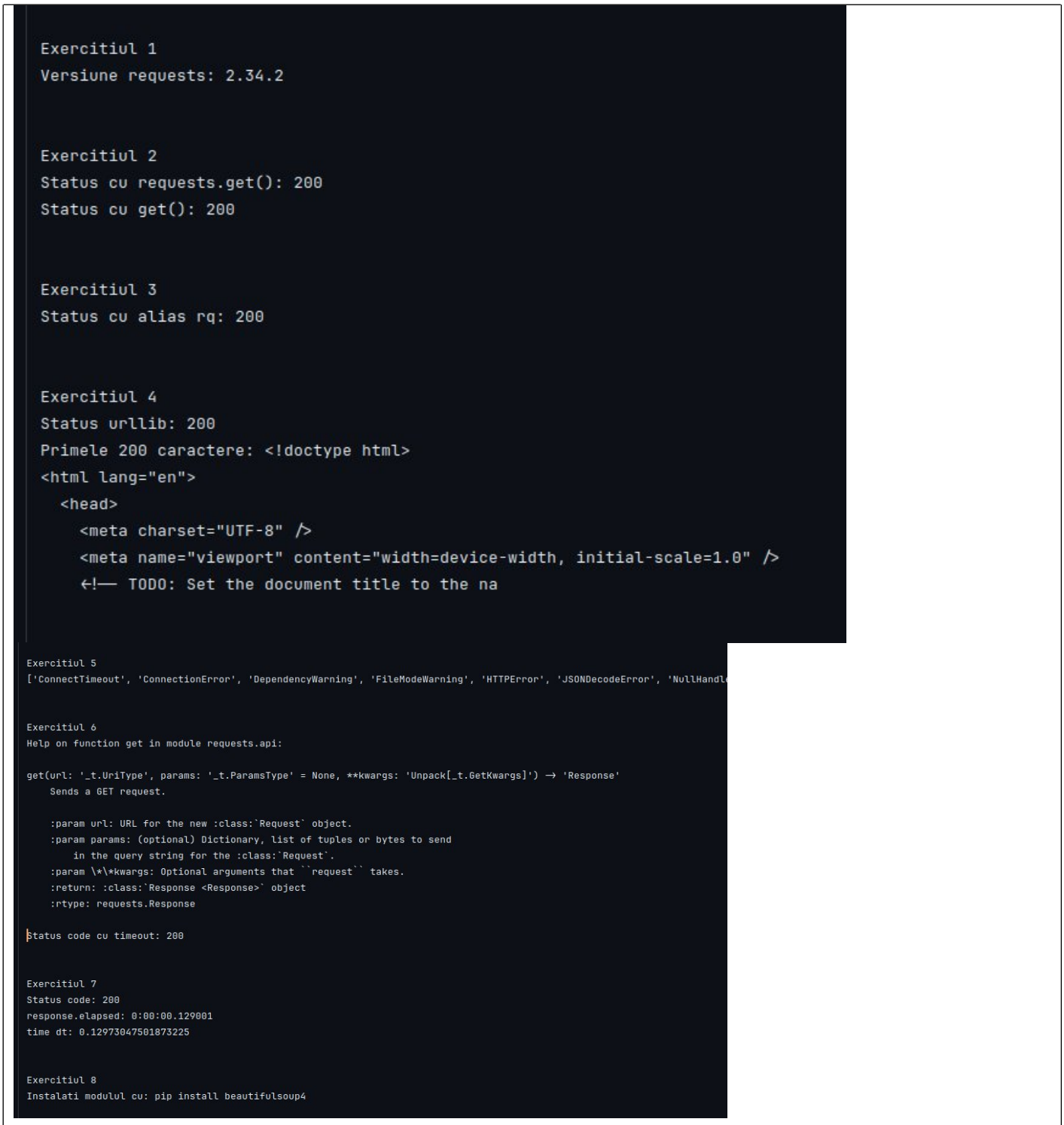


Fig. 1.1 - part1_imports.py

Partea 2: Metode și atribute ale obiectelor web (exercițiile 9–20)

Scop: învățați să folosiți instrumentele pe care obiectele le au deja. Un **atribut** este o valoare (`response.status_code`); o **metodă** este o acțiune pe care o apelați cu paranteze (`response.json()`). Fișier: `part2_methods.py`.

Exercițiul 9. Elementele de bază ale răspunsului. Descărcați pagina principală și afișați `response.status_code`, `response.ok`, `response.url` și `response.encoding`. Lângă fiecare linie, scrieți un comentariu care spune dacă este un atribut sau o metodă.

Exercițiul 10. Tratarea erorilor cu o metodă. Apelați `response.raise_for_status()` pe pagina principală. Apoi trimiteți o cerere către `BASE_URL + "/this-page-does-not-exist"` și prindeți excepția `requests.HTTPError`. Afișați un mesaj prietenos în loc să lăsați programul să se oprească cu eroare.

Exercițiul 11. Toate antetele. Parcurgeți `response.headers.items()` și afișați fiecare antet în forma `Nume: valoare`.

Exercițiul 12. Metode de dicționar. Folosiți `response.headers.get()` pentru a afișa antetele `Server` și `Content-Type`, cu `"lipsește"` ca valoare implicită. Apoi încercați `response.headers.get("content-type")`, cu litere mici. Ce observați?

Exercițiul 13. Înlănțuirea metodelor de șir. Numărați de câte ori apare cuvântul `cyber` pe pagina principală, cu `response.text.lower().count("cyber")`. Într-un comentariu, explicați de ce puteți înlănțui `.lower()` și `.count()`.

Exercițiul 14. Găsirea titlului. Folosiți `.find()` și felierea (slicing) pe `response.text` pentru a extrage textul dintre `<title>` și `</title>`. Curățați-l cu `.strip()` și afișați-l.

> **Indiciu:** `start = html.find("<title>") + len("<title>")`

Exercițiul 15. Numărarea liniilor. Împărțiți codul HTML în linii cu `.splitlines()`. Afișați câte linii sunt și lungimea celei mai lungi linii, folosind `max(lines, key=len)`.

Exercițiul 16. Verificarea HTTPS. Folosiți `.startswith()` pentru a verifica dacă `response.url` începe cu `https://`. Afișați `Conexiune securizată` sau `Conexiune nesecurizată`.

Exercițiul 17. Urmărirea redirecționărilor. Trimiteți o cerere către `http://cybercor.org` (HTTP simplu, fără S). Parcurgeți `response.history` și afișați codul de stare și URL-ul fiecărei redirecționări, apoi URL-ul final.

Exercițiul 18. HEAD versus GET. Trimiteți o cerere către pagina principală cu `requests.head()` și una cu `requests.get()`. Comparați `len(response.content)` pentru ambele și explicați diferența într-un comentariu.

Exercițiul 19. Cookie-uri. Parcurgeți `response.cookies` și afișați, pentru fiecare cookie, atributele `.name` și `.secure`. Dacă nu există cookie-uri, afișați `Niciun cookie setat`.

Exercițiul 20. Sesiuni. Creați un obiect `requests.Session()` și setați un `User-Agent` personalizat cu `session.headers.update({"User-Agent": "WebLab-<numele vostru>"})`. Trimiteți o cerere către `ECHO_URL + "/headers"` și confirmați în rezultat că antetul a fost trimis.

> **Verificați-vă:** atât `response.text`, cât și `response.json()` vă dau corpul răspunsului. De ce unul are paranteze, iar celălalt nu?

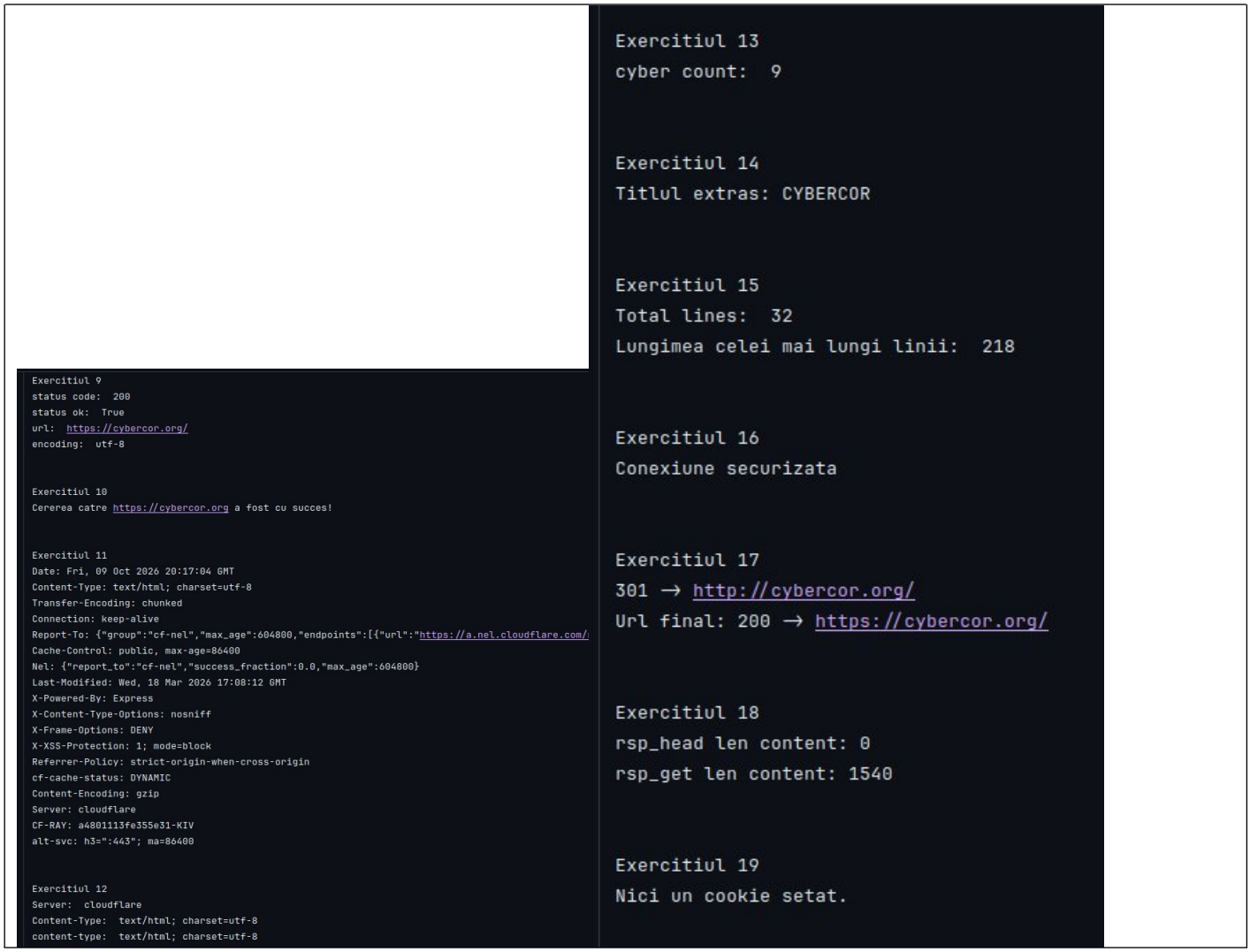




Fig. 1.2 - part2_methods.py

Partea 3: Scrierea propriilor funcții (exercițiile 21–34)

Scop: organizați codul în funcții reutilizabile. De acum înainte, fiecare exercițiu este o funcție pe care o apelați cel puțin o dată. Fișier: `part3_functions.py`.

```
def get_status(url):
    """Returnează codul de stare HTTP pentru adresa url."""
    response = requests.get(url, timeout=TIMEOUT)
    return response.status_code
```

Exercițiul 21. Prima funcție. Scrieți `fetch(url)`, care returnează obiectul răspuns. Apelați-o pentru `BASE_URL` și afișați codul de stare.

Exercițiul 22. Returnarea unei singure valori. Scrieți `get_status(url)`, care returnează doar codul de stare. Apelați-o pentru `/`, `/robots.txt` și `/sitemap.xml`, cu `time.sleep(1)` între apeluri.

Exercițiul 23. Parametri impliciți. Modificați funcția în `fetch(url, timeout=10)`. Apelați-o o dată cu valoarea implicită și o dată cu `timeout=3`.

Exercițiul 24. O singură sarcină pe funcție. Scrieți `get_title(html)`, care primește un șir HTML și returnează titlul paginii, refolosind logica din exercițiul 14. Observați că această funcție nu face nicio operație de rețea: descărcarea și analiza sunt acum separate.

Exercițiul 25. Docstring-uri. Adăugați un docstring funcției `get_title` și afișați-l cu `help(get_title)`.

Exercițiul 26. Adnotări de tip. Adăugați adnotări de tip (type hints) funcțiilor, de exemplu `def get_status(url: str) -> int:`. Apelați `get_status(123)` și observați ce se întâmplă. Vă opresc adnotările de tip?

Exercițiul 27. Fără opriri neașteptate. Scrieți `page_exists(url)`, care returnează `True` sau `False`. Folosiți `try` / `except requests.RequestException`, astfel încât funcția să returneze `False` în loc să se oprească cu eroare, chiar și pentru un URL precum `https://this-domain-does-not-exist.invalid`.

Exercițiul 28. Returnarea unui dicționar. Scrieți `check_paths(base, paths)`, care primește o listă precum `["/", "/robots.txt", "/sitemap.xml"]` și returnează un dicționar `{path: status_code}`.

Exercițiul 29. Argumente cu nume. Scrieți `get_header(url, name, default="lipsește")`, care returnează valoarea unui singur antet. Apelați-o cu argumente cu nume: `get_header(BASE_URL, name="Server")`.

Exercițiul 30. Antete de securitate. Scrieți `security_headers(url)`, care verifică prezența acestor cinci antete și returnează `{antet: True/False}`:

- `Strict-Transport-Security`
- `Content-Security-Policy`
- `X-Frame-Options`
- `X-Content-Type-Options`
- `Referrer-Policy`

Exercițiul 31. Funcții care apelează funcții. Scrieți `score_headers(results)`, care primește dicționarul din exercițiul 30 și returnează un șir precum `"3/5"`. Apelați-le împreună: `score_headers(security_headers(BASE_URL))`.

Exercițiul 32. Două funcții, o sarcină. Scrieți `fetch_robots(base)`, care returnează textul fișierului `/robots.txt` sau `None` dacă fișierul lipsește. Apoi scrieți `disallowed_paths(robots_text)`, care returnează o listă cu toate valorile `Disallow:`. Asigurați-vă că funcționează și atunci când primește `None`.

Exercițiul 33. Oricâte argumente. Scrieți `response_times(*urls)`, care returnează un dicționar `{url: secunde}`. Apelați-o cu două URL-uri.

Exercițiul 34. Argumente cu nume colectate. Scrieți `log(message, **details)`, care afișează mesajul urmat de fiecare detaliu. Exemplu: `log("verificat", url=BASE_URL, status=200)` afișează `verificat | url=https://cybercor.org | status=200`.

> **Verificați-vă:** care este diferența dintre a afișa o valoare cu `print()` în interiorul unei funcții și a o returna cu `return`?

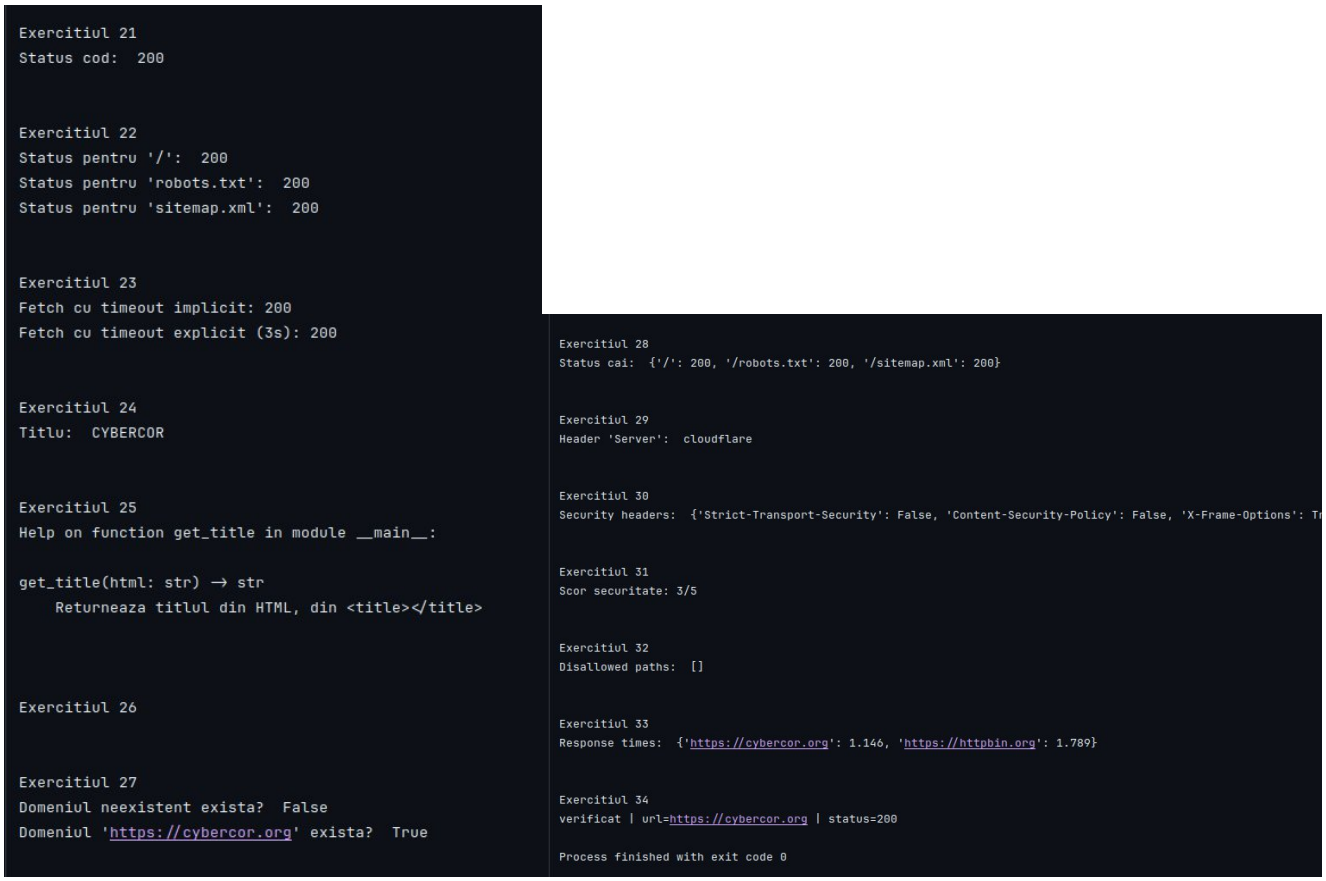


Fig. 1.3 - part3_functions.py

Partea 4: Alte module din biblioteca standard (exercițiile 35–43)

Scop: combinați mai multe importuri pentru a rezolva probleme mai mari. Puneți fiecare soluție într-o funcție. Fișier: `part4_modules.py`.

Modul	La ce îl folosiți aici
`urllib.parse`	Descompunerea și compunerea URL-urilor
`re`	Căutarea de tipare în text
`html.parser`	Citirea corectă a codului HTML

Modul	La ce îl folosiți aici
`hashlib`	Calcularea amprentei unei pagini
`json`	Salvarea datelor într-un fișier
`socket`	Aflarea adreselor IP și deschiderea conexiunilor
`ssl` + `datetime`	Citirea datei de expirare a unui certificat

Exercițiul 35. Descompunerea unui URL. Folosiți `urllib.parse.urlparse()` pe `https://cybercor.org/path?x=1#top`. Afișați `.scheme`, `.netloc`, `.path`, `.query` și `.fragment`.

Exercițiul 36. Compunerea URL-urilor. Folosiți `urllib.parse.urljoin()` pentru a transforma legăturile relative `/about`, `contact.html` și `../index.html` în URL-uri complete, pornind de la `BASE_URL`.

Exercițiul 37. Extragerea legăturilor. Folosiți `re.findall(r'href="([^"]+)"', html)` pentru a obține toate legăturile de pe pagina principală. Puneți codul într-o funcție `extract_links(html)`, care returnează legăturile fără duplicate.

Exercițiul 38. Interne sau externe? Scrieți `split_links(links, domain)`, care returnează două liste: legăturile de pe același domeniu și legăturile către alte site-uri. Folosiți mai întâi `urljoin()`, apoi `urlparse().netloc`.

Exercițiul 39. Propriile metode. Creați o clasă care moștenește `html.parser.HTMLParser` și suprascrieți metoda `handle_starttag()`, pentru a colecta atributul `src` al fiecărui tag `<img>`.

```
from html.parser import HTMLParser

class ImageFinder(HTMLParser):
    def __init__(self):
        super().__init__()
        self.images = []

    def handle_starttag(self, tag, attrs):
        # codul vostru aici
        pass
```

> **Indiciu:** `attrs` este o listă de perechi `(nume, valoare)`, de exemplu `[("src", "/logo.png"), ("alt", "Logo")]`. Expresia `dict(attrs).get("src")` vă dă valoarea atributului `src` sau `None`.

Cum testați exercițiul 39. Testați mai întâi parserul pe un fragment HTML scris de voi, pentru care știți dinainte rezultatul corect. Abia după aceea rulați-l pe site-ul real. Adăugați acest test sub clasa voastră:

```
test_html = """
<html><body>
  <img src="/logo.png" alt="Logo">
  <IMG SRC="poza.jpg">
  <img alt="imagine fără src">
  <img src="https://cdn.example.com/banner.webp" />
  <a href="/despre">Aceasta nu este o imagine</a>
</body></html>
"""

finder = ImageFinder()
finder.feed(test_html)
print(finder.images)

assert finder.images == [
    "/logo.png",                    # imagine obișnuită
    "poza.jpg",                     # tag scris cu majuscule
    "https://cdn.example.com/banner.webp",  # tag care se închide singur
], "Parserul nu a găsit exact imaginile așteptate"
print("Testul a trecut!")
```

Dacă testul trece, parserul vostru: găsește tag-urile `<img>` indiferent de majuscule, ignoră tag-urile `<img>` fără `src`, ignoră celelalte tag-uri (de exemplu `<a>`) și păstrează ordinea din pagină. Dacă `assert` eșuează, comparați lista afișată de `print()` cu lista așteptată.

Apoi rulați parserul pe pagina reală:

```
response = requests.get(BASE_URL, timeout=TIMEOUT)
finder = ImageFinder()
finder.feed(response.text)
print(len(finder.images), "imagini găsite")
for src in finder.images:
    print(src)
```

Verificați rezultatul în două moduri: comparați numărul găsit cu `response.text.lower().count("<img")` și deschideți pagina în browser, apoi apăsați Ctrl+U ca să vedeți codul sursă. O listă goală nu înseamnă neapărat o greșeală: unele site-uri afișează imaginile prin CSS sau SVG, nu prin tag-uri `<img>`. Tocmai de aceea testul cu HTML cunoscut vine primul.

Exercițiul 40. Amprenta paginii. Scrieți `page_fingerprint(url)`, care returnează `hashlib.sha256(response.content).hexdigest()`. Rulați-o de două ori și comparați rezultatele. Când ar fi diferite?

Exercițiul 41. Salvarea în JSON. Salvați antetele paginii principale în `headers.json` cu `json.dump(dict(response.headers), file, indent=2)`. Apoi încărcați fișierul înapoi cu `json.load()` și afișați un antet din el.

Exercițiul 42. Interogare DNS. Scrieți `resolve(hostname)`, care returnează `socket.gethostbyname(hostname)`. Afișați adresa IP a domeniului `cybercor.org`.

Exercițiul 43. Expirarea certificatului. Scrieți `cert_days_left(hostname)`, care se conectează la portul 443, citește certificatul și returnează câte zile mai sunt până la expirarea lui.

> **Indiciu:** folosiți `ssl.create_default_context()`, `socket.create_connection((hostname, 443))`, `context.wrap_socket(sock, server_hostname=hostname)` și `getpeercert()["notAfter"]`. Convertiți data cu `ssl.cert_time_to_seconds()`.

> **Verificați-vă:** în exercițiul 39, cine apelează `handle_starttag()`? Voi nu o apelați niciodată direct.

```
Exercitiul 35
scheme:  https
netloc:  cybercor.org
path:  /path
query:  x=1
fragment:  top


Exercitiul 36
https://cybercor.org/about
https://cybercor.org/contact.html
https://cybercor.org/index.html


Exercitiul 37
Numar legaturi gasite: 1


Exercitiul 38
Legaturi interne: 1, Legaturi externe: 0


Exercitiul 39
Testul unitar pentru ImageFinder a trecut cu succes!
0 imagini gasite pe pagina reala.


Exercitiul 40
FP1: c04dd60a2651465e1aa75da82b7e1c5daf04b38c3663c6aaef38efb6e4be89e4
FP2: c04dd60a2651465e1aa75da82b7e1c5daf04b38c3663c6aaef38efb6e4be89e4
Sunt ambele fingerprinturi egale?  True
```

```
Exercitiul 41
Content-Type extras din json: text/html; charset=utf-8


Exercitiul 42
IP pentru https://cybercor.org: 104.21.91.71


Exercitiul 43
Zile ramase certificat SSL: 54
```

Fig. 1.4 - part4_modules.py

Partea 5: Construirea propriului modul (exercițiile 44–50)

Scop: transformați funcțiile într-un modul pe care alte programe îl pot importa, la fel ca `requests`. La final, directorul vostru ar trebui să arate astfel:

```
web_lab/
├── webtools.py    # modulul vostru: toate funcțiile reutilizabile
├── main.py        # programul care importă webtools
├── report.csv     # creat în exercițiul 49
└── report.json    # creat în exercițiul 50
```

Exercițiul 44. Crearea unui modul. Mutați `fetch`, `get_status`, `get_title` și `security_headers` într-un fișier nou, `webtools.py`. În `main.py`, scrieți `import webtools` și apelați `webtools.get_title(webtools.fetch(BASE_URL).text)`.

Exercițiul 45. Importarea anumitor nume. În `main.py`, treceți la `from webtools import get_title, security_headers`. Într-un comentariu, explicați ce s-ar întâmpla dacă `main.py` ar avea și o funcție proprie numită `get_title`.

Exercițiul 46. Blocul main. Adăugați acest bloc la finalul fișierului `webtools.py`:

```
if __name__ == "__main__":
    print("Autotest:", get_status("https://cybercor.org"))
```

Rulați `python webtools.py`, apoi `python main.py`. Explicați de ce autotestul rulează doar în primul caz.

Exercițiul 47. Constante de modul. Adăugați constanta `DEFAULT_HEADERS = {"User-Agent": "WebLab-<numele vostru>"}` în `webtools.py` și folosiți-o în `fetch()`. În `main.py`, importați constanta și afișați-o.

Exercițiul 48. Argumente din linia de comandă. Folosiți `argparse` în `main.py`, astfel încât programul să ruleze ca `python main.py https://cybercor.org`. Transmiteți argumentul URL funcțiilor voastre, în loc să folosiți `BASE_URL`.

Exercițiul 49. Un raport CSV. Folosiți modulul `csv` pentru a salva rezultatul funcției `check_paths()` în `report.csv`, cu coloanele `path`, `status` și `checked_at`. Pentru oră, folosiți `datetime.now().isoformat()`.

Exercițiul 50. Proiect final: raportu l site-ului. Scrieți `site_report(url)` în `webtools.py`. Funcția trebuie să folosească funcțiile voastre pentru a colecta toate informațiile de mai jos, să le afișeze și să le salveze în `report.json`:

1. Codul de stare și URL-ul final
2. Titlul paginii
3. Adresa IP
4. Lanțul de redirecționări pornind de la `http://`
5. Scorul antetelor de securitate, de exemplu `3/5`
6. Zilele rămase până la expirarea certificatului
7. Numărul de legături interne și externe
8. Căile interzise din `robots.txt`

Un exemplu de rezultat (valorile voastre vor fi diferite):

```
=== Raport site: https://cybercor.org ===
Cod de stare:     200
Titlu:            CYBERCOR
Adresă IP:        203.0.113.10
Redirecționări:   301 -> https://cybercor.org/
Scor securitate:  3/5
Certificat:       74 de zile rămase
Legături:         38 interne, 12 externe
Căi interzise:    /admin, /private
Salvat în report.json
```

> **Bonus:** instalați `beautifulsoup4` și rescrieți `get_title()` și `extract_links()` folosind această bibliotecă. Comparați cât de mult mai scurte și mai sigure devin.

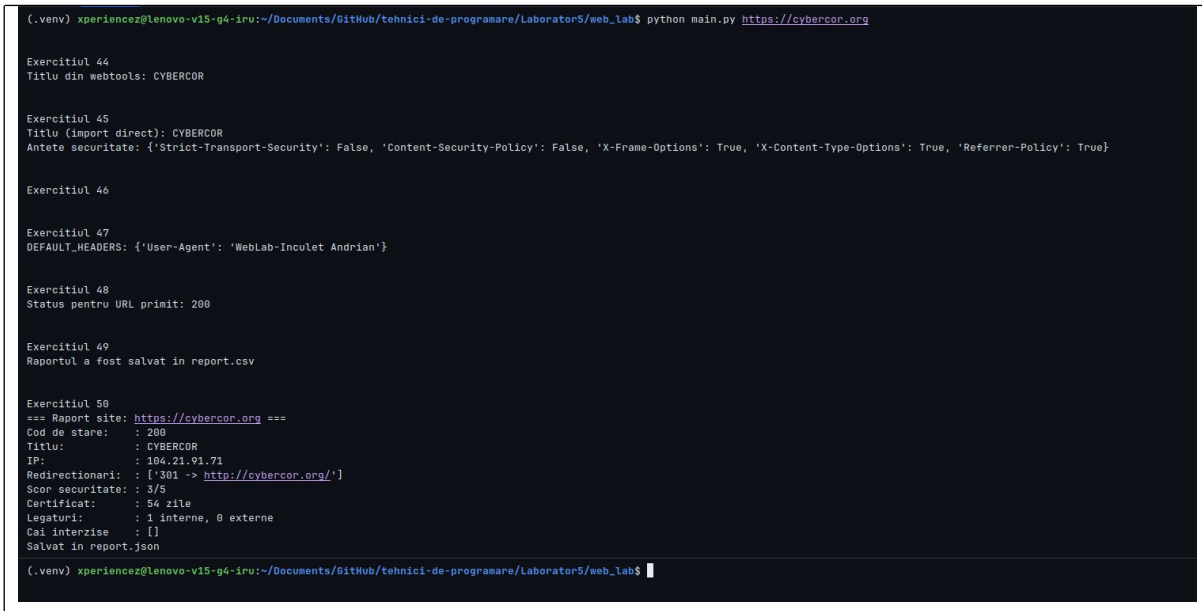


Fig. 1.5 – web_lab/main.py

CONCLUZII

In cadrul acestui laborator am invatat cum sa folosesc Python ca sa interactionez cu siteuri si sa prelucrez date.

Am efectuat urmatoarele:

Am inteles diferenta dintre modulele built-in in Python ‘urllib’ si cele instalate separat ‘requests’, dar si cum sa le import corect.

Am invatat diferenta dintre atribute - informatii salvate, cum ar fi codul de stare sau URL-ul; si metode - actiuni, cum ar fi transformarea in JSON sau verificarea erorilor.

Am creat functii simple care fac o singura sarcina si am adaugat tratarea erorilor pentru ca programul sa nu se opreasca daca un site nu functioneaza.

Am folosit instrumente din Python pentru a descompune URL-uri, a extrage link-uri si imagini din HTML, a afla adresa IP a unui site si a calcula cate zile mai sunt valabile certificatele de securitate.

Am mutat toate functiile intr-un fisier separat webtools.py si am creat un program principal care analizeaza un site si salveaza raportul final in fisiere CSV si JSON.

Aceasta lucrare m-a ajutat sa inteleg cum functioneaza cererile pe web si cum pot scrie un cod Python simplu si usor.

ANEXE

part1_imports.py

```
# Laborator:  functii, metode si importuri pe web
# Student: Inculet Andrian
import time

import requests
import requests as rq
from requests import get
import urllib.request

BASE_URL = "https://cybercor.org"
ECHO_URL = "https://httpbin.org"
TIMEOUT = 10  # secunde

## ex 1
print("\n\nExercitiul 1")
print("Versiune requests:", requests.__version__)
# requests are un nevoie de pip install fiindca e o blbioteca 3rd party
# urllib e un modul built-in in python


## ex 2
print("\n\nExercitiul 2")
status_code = requests.get(BASE_URL, timeout=TIMEOUT).status_code
print("Status cu requests.get():", status_code)

time.sleep(1)
status_code = get(BASE_URL, timeout=TIMEOUT).status_code
print("Status cu get():", status_code)
# avantaj la import requests: codul e clar, si evita conflictele cu alte functii
# avantaj la from requests import get: codul e mai scurt si mai usor de scris


## ex 3
print("\n\nExercitiul 3")
time.sleep(1)
pagina = rq.get(BASE_URL, timeout=TIMEOUT)
print("Status cu alias rq:", pagina.status_code)
# aliasul face codul mai usor de citit cand numele modulului e foarte lung sau
frecvent utlizat
# aliasul face codul mai greu de citit cand se utlizeaza aliasuri scurte sau prea
multe aliasuri in acelasi fisier


## ex 4
print("\n\nExercitiul 4")
headers = {"User-Agent": "laborator"}
request = urllib.request.Request(BASE_URL, headers=headers)

time.sleep(1)
with urllib.request.urlopen(request, timeout=TIMEOUT) as raspuns:
    print("Status urllib:", raspuns.status)
    print("Primele 200 caractere:", raspuns.read().decode("utf-8")[:200])


## ex 5
print("\n\nExercitiul 5")
print(dir(requests))
# get - functie - trimite o cerere http get
# Response - clasa - obiect returnat de cereri
```

```
# exceptions - modul - pachet pentru tratarea erorilor http sau retea


## ex 6
print("\n\nExercitiul 6")
help(requests.get)
# parametrul care seteaza timpul maxim de asteptare este `timeout`.

time.sleep(1)
raspuns = requests.get(BASE_URL, timeout=TIMEOUT)
print("Status code cu timeout:", raspuns.status_code)


## ex 7
print("\n\nExercitiul 7")
time.sleep(1)
start = time.perf_counter()
raspuns = requests.get(BASE_URL, timeout=TIMEOUT)
end = time.perf_counter()
dt = end - start

print("Status code:", raspuns.status_code)
print("response.elapsed:", raspuns.elapsed)
print("time dt:", dt)


## ex 8
print("\n\nExercitiul 8")
try:
    import bs4
    print("Modulul bs4 este deja instalat!")
except ImportError:
    print("Instalati modulul cu: pip install beautifulsoup4")


# un modul e un fisier py
# un packet e un director cu fisiere py
# o biblioteca ii o colectie de pachete si module p/u o functionalitate extinsa
```

part2_methods.py

```
# Laborator:  functii, metode si importuri pe web
# Student: Inculet Andrian

import time
import requests
import requests as rq

BASE_URL = "https://cybercor.org"
ECHO_URL = "https://httpbin.org"
TIMEOUT = 10  # secunde

## ex 9
print("\n\nExercitiul 9")
raspuns = rq.get(BASE_URL, timeout=TIMEOUT, headers={"User-Agent": "laborator"})

print("status code: ", raspuns.status_code)  # atribut
print("status ok: ", raspuns.ok)             # metoda
```

```
print("url: ", raspuns.url)                  # atribut
print("encoding: ", raspuns.encoding)        # atribut


## ex 10
print("\n\nExercitiul 10")
try:
    time.sleep(1)
    raspuns_good = rq.get(BASE_URL, timeout=TIMEOUT, headers={"User-Agent":
"laborator"})
    raspuns_good.raise_for_status()
    print(f"Cererea catre {BASE_URL} a fost cu succes!")

    time.sleep(1)
    raspuns_bad = rq.get(BASE_URL + "/this-page-does-not-exist", timeout=TIMEOUT,
headers={"User-Agent": "laborator"})
    raspuns_bad.raise_for_status()
except requests.HTTPError as err:
    print(f"Pagina nu exista sau a returnat eroare: {err}")


## ex 11
print("\n\nExercitiul 11")
time.sleep(1)
raspuns = rq.get(BASE_URL, timeout=TIMEOUT, headers={"User-Agent": "laborator"})

for key, value in raspuns.headers.items():
    print(f"{key}: {value}")


## ex 12
print("\n\nExercitiul 12")
print("Server: ", raspuns.headers.get("Server", "lipseste"))
print("Content-Type: ", raspuns.headers.get("Content-Type", "lipseste"))
print("content-type: ", raspuns.headers.get("content-type", "lipseste"))
# response.headers e case insensitive, deci ambele returneaza aceiasi valoare


## ex 13
print("\n\nExercitiul 13")
print("cyber count: ", raspuns.text.lower().count("cyber"))
# metodele se pot inlantui fiindca .lower() returneaza un nou str care poate fi apelat
de .count()



## ex 14
print("\n\nExercitiul 14")
start = raspuns.text.find("<title>") + len("<title>")
end = raspuns.text.find("</title>")

titlu = raspuns.text[start:end].strip()
print("Titlul extras:", titlu)


## ex 15
```

```
print("\n\nExercitiul 15")
lines = raspuns.text.splitlines()

total_lines = len(lines)
longest_line = max(lines, key=len) if lines else ""
max_len = len(longest_line)

print("Total lines: ", total_lines)
print("Lungimea celei mai lungi linii: ", max_len)



## ex 16
print("\n\nExercitiul 16")
if raspuns.url.startswith("https://"):
    print("Conexiune securizata")
else:
    print("Conexiune nesecurizata")



## ex 17
print("\n\nExercitiul 17")
time.sleep(1)
raspuns_redir = rq.get("http://cybercor.org", timeout=TIMEOUT, headers={"User-Agent":
"laborator"})
for rsp in raspuns_redir.history:
    print(f"{rsp.status_code} -> {rsp.url}")

print(f"Url final: {raspuns_redir.status_code} -> {raspuns_redir.url}")



## ex 18
print("\n\nExercitiul 18")
time.sleep(1)
rsp_get = rq.get(BASE_URL, timeout=TIMEOUT, headers={"User-Agent": "laborator"})
time.sleep(1)
rsp_head = rq.head(BASE_URL, timeout=TIMEOUT, headers={"User-Agent": "laborator"})

print(f"rsp_head len content: {len(rsp_head.content)}")
print(f"rsp_get len content: {len(rsp_get.content)}")
# cererea head solicit doar headerele de raspuns, fara content
# cererea get descarca headerele si contentul paginii (html)



## ex 19
print("\n\nExercitiul 19")
time.sleep(1)
raspuns_cookie = rq.get(BASE_URL, timeout=TIMEOUT, headers={"User-Agent":
"laborator"})
if raspuns_cookie.cookies:
    for cookie in raspuns_cookie.cookies:
        print(f"cookie name: {cookie.name}; cookie secure: {cookie.secure}")
else:
    print("Nici un cookie setat.")



## ex 20
```

```
print("\n\nExercitiul 20")
session = requests.Session()
session.headers.update({"User-Agent": "laborator"})

time.sleep(1)
raspuns_session = session.get(ECHO_URL + "/headers", timeout=TIMEOUT)
print("Raspuns de la httpbin headers:")
print(raspuns_session.text)
# response.text e un atribut care contine textul decodat al raspunsului
# response.json() e o metoda care face json parsing si returneaza un dict/lista python
```

part3_functions.py

```
# Laborator:  functii, metode si importuri pe web
# Student: Inculet Andrian

import time
import requests

BASE_URL = "https://cybercor.org"
ECHO_URL = "https://httpbin.org"
TIMEOUT = 10  # secunde

## ex 21
print("\n\nExercitiul 21")
def fetch(url: str, timeout: int = TIMEOUT) -> requests.Response:
    """Efectueaza o cerere GET cu pauza de 1 secunda si User-Agent setat."""
    time.sleep(1)
    return requests.get(url, timeout=timeout, headers={'User-Agent': 'laborator'})

print("Status cod: ", fetch(BASE_URL).status_code)



## ex 22
print("\n\nExercitiul 22")
def get_status(url: str) -> int:
    """Returneaza codul de stare HTTP pentru adresa url."""
    return fetch(url).status_code

print("Status pentru '/': ", get_status(BASE_URL))
print("Status pentru 'robots.txt': ", get_status(BASE_URL + "/robots.txt"))
print("Status pentru 'sitemap.xml': ", get_status(BASE_URL + "/sitemap.xml"))



## ex 23
print("\n\nExercitiul 23")
print("Fetch cu timeout implicit:", fetch(BASE_URL).status_code)
print("Fetch cu timeout explicit (3s):", fetch(BASE_URL, timeout=3).status_code)



## ex 24
print("\n\nExercitiul 24")
def get_title(html: str) -> str:
    """Returneaza titlul din HTML, din <title></title>"""
    start = html.find("<title>")
    if start == -1:
        return ""
```

```
    start += len("<title>")
    end = html.find("</title>", start)
    if end == -1:
        return ""
    return html[start:end].strip()

print("Titlu: ", get_title(fetch(BASE_URL).text))


## ex 25
print("\n\nExercitiul 25")
help(get_title)


## ex 26
print("\n\nExercitiul 26")
# adnotarile de tip sunt cu scop de informare
# eg get_status(123) va esua din cauza tipului de date incorect


## ex 27
print("\n\nExercitiul 27")
def page_exists(url: str) -> bool:
    """Verifica daca o pagina exista si trateaza exceptiile."""
    try:
        return get_status(url) == 200
    except requests.RequestException:
        return False

print("Domeniul neexistent exista? ", page_exists("https://this-domain-does-not-
exist.invalid"))
print(f"Domeniul '{BASE_URL}' exista? ", page_exists(BASE_URL))


## ex 28
print("\n\nExercitiul 28")
def check_paths(base: str, paths: list[str]) -> dict[str, int]:
    """Verifica o lista de cai relative si returneaza un dictionar {path:
status_code}."""
    results = {}
    for path in paths:
        results[path] = get_status(base + path)
    return results

print("Status cai: ", check_paths(BASE_URL, ["/", "/robots.txt", "/sitemap.xml"]))


## ex 29
print("\n\nExercitiul 29")
def get_header(url: str, name: str, default: str = "lipseste") -> str:
    """Returneaza valoarea unui antet specific de la un URL."""
    return fetch(url).headers.get(name, default)

print("Header 'Server': ", get_header(BASE_URL, name="Server"))
```

```
## ex 30
print("\n\nExercitiul 30")
def security_headers(url: str) -> dict[str, bool]:
    """Verifica prezenta principalelor 5 antete de securitate."""
    headers = fetch(url).headers
    return {
        "Strict-Transport-Security": "Strict-Transport-Security" in headers,
        "Content-Security-Policy": "Content-Security-Policy" in headers,
        "X-Frame-Options": "X-Frame-Options" in headers,
        "X-Content-Type-Options": "X-Content-Type-Options" in headers,
        "Referrer-Policy": "Referrer-Policy" in headers
    }

print("Security headers: ", security_headers(BASE_URL))



## ex 31
print("\n\nExercitiul 31")
def score_headers(results: dict[str, bool]) -> str:
    """Calculeaza scorul antetelor de securitate prezente."""
    score = sum(1 for present in results.values() if present)
    return f"{score}/{len(results)}"

print("Scor securitate:", score_headers(security_headers(BASE_URL)))



## ex 32
print("\n\nExercitiul 32")
def fetch_robots(base: str) -> str | None:
    """Returneaza textul fisierului /robots.txt sau None daca nu exista."""
    robots_url = base.rstrip("/") + "/robots.txt"
    if page_exists(robots_url):
        return fetch(robots_url).text
    return None

def disallowed_paths(robots_text: str | None) -> list[str]:
    """Extrage caile blocate (Disallow:) din continutul robots.txt."""
    if not robots_text:
        return []
    paths = []
    for line in robots_text.splitlines():
        line = line.strip()
        if line.lower().startswith("disallow:"):
            parts = line.split(":", 1)
            if len(parts) > 1:
                path = parts[1].strip()
                if path:
                    paths.append(path)
    return paths

print("Disallowed paths: ", disallowed_paths(fetch_robots(BASE_URL)))



## ex 33
print("\n\nExercitiul 33")
def response_times(*urls: str) -> dict[str, float]:
    """Masoara timpul de raspuns pentru un numar variabil de URL-uri."""
```

```
    times = {}
    for url in urls:
        start = time.time()
        fetch(url, timeout=TIMEOUT)
        end = time.time()
        times[url] = round(end - start, 3)
    return times

print("Response times: ", response_times(BASE_URL, ECHO_URL))


## ex 34
print("\n\nExercitiul 34")
def log(message: str, **details) -> None:
    """Afiseaza un mesaj urmat de detalii cheie-valoare."""
    details_str = " | ".join(f"{k}={v}" for k, v in details.items())
    print(f"{message} | {details_str}")

log("verificat", url=BASE_URL, status=200)

# cand se afiseaza o valoare intr-o metoda, se face print() si se returneaza None
(depinde de caz_
# cand se returneaza inapoi ea poate fi salvata intr-o variabila sau prelucrata
```

part4_modules.py

```
# Laborator:  functii, metode si importuri pe web
# Student: Inculet Andrian
import hashlib
import json
import re
import socket
import ssl
import time
from html.parser import HTMLParser
import urllib.parse
import requests

BASE_URL = "https://cybercor.org"
ECHO_URL = "https://httpbin.org"
TIMEOUT = 10  # secunde

def fetch(url: str) -> requests.Response:
    time.sleep(1)
    return requests.get(url, timeout=TIMEOUT, headers={"User-Agent": "laborator"})

## ex 35
print("\n\nExercitiul 35")
rezultat = urllib.parse.urlparse("https://cybercor.org/path?x=1#top")

print("scheme: ", rezultat.scheme)
print("netloc: ", rezultat.netloc)
print("path: ", rezultat.path)
print("query: ", rezultat.query)
print("fragment: ", rezultat.fragment)
```

```
## ex 36
print("\n\nExercitiul 36")
print(urllib.parse.urljoin(BASE_URL, "/about"))
print(urllib.parse.urljoin(BASE_URL, "contact.html"))
print(urllib.parse.urljoin(BASE_URL, "../index.html"))



## ex 37
print("\n\nExercitiul 37")
def extract_links(html: str) -> set[str]:
    """Extrage toate legaturile href dintr-un HTML fara duplicate."""
    return set(re.findall(r'href="([^"]+)"', html))

html_content = fetch(BASE_URL).text
print("Numar legaturi gasite:", len(extract_links(html_content)))



## ex 38
print("\n\nExercitiul 38")
def split_links(links: set[str], base_url: str) -> tuple[list[str], list[str]]:
    """Separa linkurile interne si externe folosind urljoin si urlparse."""
    same_domain = []
    diff_domains = []
    base_netloc = urllib.parse.urlparse(base_url).netloc

    for link in links:
        full_url = urllib.parse.urljoin(base_url, link)
        link_netloc = urllib.parse.urlparse(full_url).netloc
        if link_netloc == base_netloc:
            same_domain.append(full_url)
        else:
            diff_domains.append(full_url)

    return same_domain, diff_domains

interne, externe = split_links(extract_links(html_content), BASE_URL)
print(f"Legaturi interne: {len(interne)}, Legaturi externe: {len(externe)}")



## ex 39
print("\n\nExercitiul 39")
class ImageFinder(HTMLParser):
    """Parser HTML pentru colectarea atributelor src ale tag-urilor img."""
    def __init__(self):
        super().__init__()
        self.images = []

    def handle_starttag(self, tag, attrs):
        if tag.lower() == "img":
            src = dict(attrs).get("src")
            if src is not None:
                self.images.append(src)

test_html = """
<html><body>
  <img src="/logo.png" alt="Logo">
```

```
  <IMG SRC="poza.jpg">
  <img alt="imagine fara src">
  <img src="https://cdn.example.com/banner.webp" />
  <a href="/despre">Aceasta nu este o imagine</a>
</body></html>
"""

finder = ImageFinder()
finder.feed(test_html)
assert finder.images == [
    "/logo.png",
    "poza.jpg",
    "https://cdn.example.com/banner.webp",
], "Parserul nu a gasit exact imaginile asteptate"
print("Testul unitar pentru ImageFinder a trecut cu succes!")



finder_real = ImageFinder()
finder_real.feed(html_content)
print(f"{len(finder_real.images)} imagini gasite pe pagina reala.")



## ex 40
print("\n\nExercitiul 40")
def page_fingerprint(url: str) -> str:
    """Calculeaza amprenta SHA256 a continutului paginii."""
    return hashlib.sha256(fetch(url).content).hexdigest()

fp1 = page_fingerprint(BASE_URL)
fp2 = page_fingerprint(BASE_URL)

print("FP1:", fp1)
print("FP2:", fp2)
print("Sunt ambele fingerprinturi egale? ", fp1 == fp2)
# amprentele ar fi diferite daca continutul se modifica intre cererile efectuate (eg.
banere, timestampuri, etc etc)



## ex 41
print("\n\nExercitiul 41")
response = fetch(BASE_URL)
with open("headers.json", "w", encoding="utf-8") as file:
    json.dump(dict(response.headers), file, indent=2)

with open("headers.json", "r", encoding="utf-8") as file:
    headers_loaded = json.load(file)

print("Content-Type extras din json:", headers_loaded.get("Content-Type"))



## ex 42
print("\n\nExercitiul 42")
def resolve(hostname: str) -> str:
    """Returneaza adresa IP a unui nume de gazda."""
    clean_host = urllib.parse.urlparse(hostname).netloc or hostname
    return socket.gethostbyname(clean_host)
```

```
print(f"IP pentru {BASE_URL}:", resolve(BASE_URL))


## ex 43
print("\n\nExercitiul 43")
def cert_days_left(hostname: str) -> int:
    """Calculeaza numarul de zile ramase pana la expirarea certificatului SSL."""
    clean_host = urllib.parse.urlparse(hostname).netloc or hostname
    context = ssl.create_default_context()

    with socket.create_connection((clean_host, 443), timeout=TIMEOUT) as sock:
        with context.wrap_socket(sock, server_hostname=clean_host) as ssock:
            cert = ssock.getpeercert()

    expiration = cert.get("notAfter")
    exp_seconds = ssl.cert_time_to_seconds(expiration)
    return int((exp_seconds - time.time()) / (24 * 60 * 60))

print(f"Zile ramase certificat SSL:", cert_days_left(BASE_URL))
```

webtools.py

```
# Laborator:  functii, metode si importuri pe web
# Student: Inculet Andrian
import json
import re
import socket
import ssl
import time
import urllib.parse
import requests

BASE_URL = "https://cybercor.org"
ECHO_URL = "https://httpbin.org"
TIMEOUT = 10  # secunde

DEFAULT_HEADERS = {"User-Agent": "WebLab-Inculet Andrian"}

def fetch(url: str, timeout: int = TIMEOUT) -> requests.Response:
    time.sleep(1)
    return requests.get(url, timeout=timeout, headers=DEFAULT_HEADERS)

def get_status(url: str) -> int:
    return fetch(url).status_code

def get_title(html: str) -> str:
    start = html.find("<title>")
    if start == -1:
        return ""
    start += len("<title>")
    end = html.find("</title>", start)
    if end == -1:
        return ""
    return html[start:end].strip()
```

```
def security_headers(url: str) -> dict[str, bool]:
    """Verifica prezenta celor 5 antete de securitate esentiale."""
    headers = fetch(url).headers
    return {
        "Strict-Transport-Security": "Strict-Transport-Security" in headers,
        "Content-Security-Policy": "Content-Security-Policy" in headers,
        "X-Frame-Options": "X-Frame-Options" in headers,
        "X-Content-Type-Options": "X-Content-Type-Options" in headers,
        "Referrer-Policy": "Referrer-Policy" in headers
    }

def check_paths(base: str, paths: list[str]) -> dict[str, int]:
    """Verifica statusul unei liste de cai relative."""
    return {path: get_status(urllib.parse.urljoin(base, path)) for path in paths}

def extract_links(html: str) -> set[str]:
    """Extrage toate valorile din atributul href."""
    return set(re.findall(r'href="([^"]+)"', html))

def resolve(hostname: str) -> str:
    """Afla adresa IP pentru domeniul dat."""
    clean_host = urllib.parse.urlparse(hostname).netloc or hostname
    return socket.gethostbyname(clean_host)

def split_links(links: set[str], base_url: str) -> tuple[list[str], list[str]]:
    """Imparte legaturile in interne si externe."""
    same_domain = []
    diff_domains = []
    base_netloc = urllib.parse.urlparse(base_url).netloc

    for link in links:
        full_url = urllib.parse.urljoin(base_url, link)
        link_netloc = urllib.parse.urlparse(full_url).netloc
        if link_netloc == base_netloc:
            same_domain.append(full_url)
        else:
            diff_domains.append(full_url)

    return same_domain, diff_domains

def cert_days_left(hostname: str) -> int:
    """Calculeaza zilele ramase pana la expirarea certificatului SSL."""
    clean_host = urllib.parse.urlparse(hostname).netloc or hostname
    context = ssl.create_default_context()

    with socket.create_connection((clean_host, 443), timeout=TIMEOUT) as sock:
        with context.wrap_socket(sock, server_hostname=clean_host) as ssock:
            cert = ssock.getpeercert()

    expiration = cert.get("notAfter")
    exp_seconds = ssl.cert_time_to_seconds(expiration)
    return int((exp_seconds - time.time()) / (24 * 60 * 60))

def score_headers(results: dict[str, bool]) -> str:
    """Returneaza scorul antetelor (ex: 3/5)."""
```

```
    score = sum(1 for v in results.values() if v)
    return f"{score}/{len(results)}"

def page_exists(url: str) -> bool:
    """Verifica daca o pagina este accesibila (200 OK)."""
    try:
        return get_status(url) == 200
    except requests.RequestException:
        return False

def fetch_robots(base: str) -> str | None:
    """Aduce continutul robots.txt."""
    robots_url = urllib.parse.urljoin(base, "/robots.txt")
    if page_exists(robots_url):
        return fetch(robots_url).text
    return None

def disallowed_paths(robots_text: str | None) -> list[str]:
    """Extrage caile marcate cu Disallow."""
    if not robots_text:
        return []

    paths = []
    for line in robots_text.splitlines():
        line = line.strip()
        if line.lower().startswith("disallow:"):
            parts = line.split(":", 1)
            if len(parts) > 1:
                path = parts[1].strip()
                if path:
                    paths.append(path)
    return paths

def site_report(url: str) -> dict[str, object]:
    """Genereaza raportul complet al site-ului si il salveaza in report.json."""
    rsp = fetch(url)
    status = rsp.status_code
    titlu = get_title(rsp.text)
    ip = resolve(url)

    redirects = []
    http_url = url.replace("https://", "http://")
    try:
        rsp_redir = fetch(http_url)
        for r in rsp_redir.history:
            redirects.append(f"{r.status_code} -> {r.url}")
    except Exception:
        pass

    scor_sec = score_headers(security_headers(url))
    zile_cert = cert_days_left(url)

    links = extract_links(rsp.text)
    legaturi_int, legaturi_ext = split_links(links, url)

    robots = fetch_robots(url)
    cai_interzise = disallowed_paths(robots)
```

```
    rezultat = {
        "Cod de stare:    ": status,
        "Titlu:           ": titlu,
        "IP:              ": ip,
        "Redirectionari:  ": redirects,
        "Scor securitate: ": scor_sec,
        "Certificat:      ": f"{zile_cert} zile",
        "Legaturi:        ": f"{len(legaturi_int)} interne, {len(legaturi_ext)}
externe",
        "Cai interzise    ": cai_interzise,
    }

    print(f"=== Raport site: {url} ===")
    for k, v in rezultat.items():
        print(f"{k}: {v}")

    with open("report.json", "w", encoding="utf-8") as file:
        json.dump(rezultat, file, indent=2, ensure_ascii=False)

    print("Salvat in report.json")
    return rezultat

## ex 46
if __name__ == "__main__":
    print("Autotest:", get_status("https://cybercor.org"))
```

main.py

```
# Laborator:  functii, metode si importuri pe web
# Student: Inculet Andrian
import argparse
import csv
import datetime

BASE_URL = "https://cybercor.org"
ECHO_URL = "https://httpbin.org"
TIMEOUT = 10  # secunde

## ex 44
print("\n\nExercitiul 44")
import webtools
print("Titlu din webtools:", webtools.get_title(webtools.fetch(BASE_URL).text))



## ex 45
print("\n\nExercitiul 45")
from webtools import get_title, security_headers, fetch, check_paths, site_report,
DEFAULT_HEADERS

print("Titlu (import direct):", get_title(fetch(BASE_URL).text))
print("Antete securitate:", security_headers(BASE_URL))

# Explicatie: Daca main.py ar defini propria functie numita `get_title`, aceasta ar
suprascrie (shadowing)
# functia `get_title` importata anterior din webtools.
```

```
## ex 46
print("\n\nExercitiul 46")
# Blocul `if __name__ == "__main__":` din webtools.py previne executarea autotestului
la importare.
# Cand rulam `python main.py`, `__name__` in webtools este `"webtools"`, nu
`"__main__"`.


## ex 47
print("\n\nExercitiul 47")
print("DEFAULT_HEADERS:", DEFAULT_HEADERS)


## ex 48
print("\n\nExercitiul 48")
parser = argparse.ArgumentParser(description="Script de analiza web")
parser.add_argument("url", nargs="?", default=BASE_URL, help="URL-ul de analizat")

args = parser.parse_args()
target_url = args.url

response = fetch(target_url)
print("Status pentru URL primit:", response.status_code)


## ex 49
print("\n\nExercitiul 49")
paths_to_check = ["/", "/robots.txt", "/sitemap.xml"]
results_dict = check_paths(target_url, paths_to_check)

with open("report.csv", "w", newline="", encoding="utf-8") as file:
    writer = csv.DictWriter(
        file,
        fieldnames=["path", "status", "checked_at"]
    )
    writer.writeheader()
    for path, status in results_dict.items():
        writer.writerow({
            "path": path,
            "status": status,
            "checked_at": datetime.datetime.now().isoformat()
        })

print("Raportul a fost salvat in report.csv")


## ex 50
print("\n\nExercitiul 50")
site_report(target_url)
```